

Internet of Things

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University in Tashkent.

Email: minhaz.ahmed@gmail.com

Reference book



Chapter 4 and chapter 5

Outline

- ▶ Thinking About Prototyping
- ▶ Sketching
- ▶ Familiarity
- ▶ Costs versus Ease of Prototyping
- ▶ Prototypes and production
- ▶ Open Source versus Closed Source
- ▶ Micro services
- ▶ Tapping into the Community
- ▶ Prototyping Embedded devices
- ▶ References

Thinking About Prototyping

Making a prototype first has many benefits. You will inevitably come across problems in your design that you need to change and iterate. Doing this with a single object is trivial compared to modifying hundreds or thousands of products.

With the Internet of Things , we are always looking three things in parallel:

- a) The physical Thing
- b) The electronics to make the things smart
- c) The internet service that we will connect to.

Thinking About Prototyping

- ▶ Many Internet of Things projects **start with a prototyping microcontroller**, connected by wires to components on a prototyping board, such as a “breadboard”, and housed in some kind of container (e.g. old tin or a laser-cut box). **This prototype is relatively inexpensive**, but you **will most likely end up with something that is serviceable rather than polished** and that will cost more than someone would be willing to pay for it in a shop.
- ▶ **At the end of this stage, you'll have an object that works.** It may be useful for you already. It may be a talking point to show your friends. And if you are planning to move to production, it's a demonstrable product that you can use to convince yourself, **your business partners, and your investors that your idea has legs and is worth trying to sell.**

Thinking About Prototyping

Finally, the process of manufacture will iron out issues of scaling up and polish. You might substitute prototyping microcontrollers and wires with smaller chips on a printed circuit board (PCB), and pieces improvised out of 3D-printed plastic with ones commercially injection-moulding. The final product will be cheaper per unit and more professional, but will be much more expensive to change.

We could easily give a pat answer like “Use an Arduino and develop your online service in Ruby on Rails” or “Use a Raspberry Pi”. [The best choice of prototyping platform can't come from on high or from a book.](#) For one thing, your project has its own goals and requirements.

Thinking About Prototyping

► FAMILIARITY

Another option to consider is familiarity. If you can already program like a whiz in Python, for example, maybe picking a platform such as Raspberry Pi, which lets you write the code in a language you already know, would be better than having to learn Arduino from scratch.

COSTS VERSUS EASE OF PROTOTYPING

Although familiarity with a platform may be attractive in terms of ease of prototyping, it is also worth considering the relationship between the costs of a platform against the development effort that the platform demands. **This trade-off is not hard and fast, but it is beneficial if you can choose a prototyping platform in a performance/capabilities bracket similar to a final production solution.** That way, you will be less likely to encounter any surprises over the cost, or even the wholesale viability of your project, down the line .

For example, the cheapest possible way of creating an electronic device might currently be an AVR microcontroller chip, which you can purchase from a component supplier for about £3. This amount is just for the chip. For many people, this platform would not be viable for an initial prototype.

Prototype versus production

Although ease of prototyping is a major factor, perhaps the biggest obstacle to getting a project started—scaling up to building more than one device, perhaps many thousands of them—brings a whole new set of challenges and questions.

When you scale up, you may well have to think about moving to a different platform, for cost or size reasons. If you've started with a free-form, powerful programming platform, you may find that porting the code to a more restricted, cheaper, and smaller device will bring many challenges.

Open Source Versus Closed Source

► Open Source

Advantage:

- First, using open source work is often a no-risk way of getting software that has been tested, improved, and debugged by many eyes.
- Second, using open source aggressively gives your product the chance to gain mindshare.
- Your assertion, **as the creator, of your Intellectual Property rights**
- Your users' rights to freely tinker with your creation
- The democratic and open approach from hackspaces, along with that of the open source software community, has carried over into the Maker culture, resulting in a default stance of sharing designs and code.
- source files are available for **learning, re-using, and extending** into new products.

Open Source Versus Closed Source

▶ Open Source

Disadvantage:

- ▶ People might know your architecture.
- ▶ For commercialization you may have to pay the royalty.
- ▶ Some unwanted feature might be included.

Open Source Versus Closed Source

▶ Closed Source

Advantage:

- ▶ If you declared copyright on some source code or a design, someone who wants to market the same project cannot do so by simply reading your instructions and following them.
- ▶ That person would have to **instead reverse-engineer the functionality of the hardware and software.**
- ▶ You might also be able to **protect distinctive elements of the visual design with trademarks and of the software and hardware with patents.**
- ▶ If you are developing an Internet of Things device in such a context, working within the culture of the company may simply be easier, unless you are willing to try to persuade your management, marketing.

Reason for making open source

- ▶ You may gain positive comments from people who liked it.
- ▶ It acts as a public showcase of your work, which may affect your reputation and lead to new opportunities.
- ▶ People who used your work may suggest or implement features or fix bugs.
- ▶ By generating early interest in your project, you may get support and mindshare of a quality that it would be hard to pay for.
- ▶ you can use other people's free and open source contributions within your own project.
- ▶ A bug fix from someone who tried using your code in a way you had never thought of may save you hours of unpleasant debugging later.

Open Source Versus Closed Source

Disadvantage:

- ▶ people may steal your idea!
- ▶ All code might not work efficiently. Lack of reliability.
- ▶ After you release something as open source, you may still have a perceived duty to maintain and support it, or at least to answer questions

Conclusion

- ▶ Prototyping is inherently a matter of balancing trade-offs between building something that allows you to learn more about the project you are looking to build and keeping an eye on how things scale up should your experiments prove you right.
- ▶ Different tools may be suitable for different kinds users,
e.g.
 - developers,
 - researchers,
 - designers,
 - educator, and
 - hobbyists.
- ▶ As the IoT vision gradually becomes a reality, using connected-device prototypes to explore the design space will be important.

Prototyping in Embedded Devices (chp-5)

- ▶ Electronics
- ▶ Sensors
- ▶ Actuators
- ▶ Scaling Up the Electronics
- ▶ Embedded Computing Basics
- ▶ Microcontrollers
- ▶ System-on-Chips
- ▶ Choosing your platform

Electronics

Electronics, it's useful to split them into two main categories:

- ▶ **Sensors:** Sensors are the ways of getting information into your device, finding out things about your surroundings.
- ▶ **Actuators:** Actuators are the outputs for the device—the motors, lights, and so on, which let your device do something to the outside world
- ▶ Within both categories the electronic components can talk to computer in a number of ways

Electronics

- ▶ The simplest is through digital I/O, which has only two states: a button can either be pressed or not; or an LED can be on or off. These states are usually connected via general-purpose input/output (GPIO) pins and map a digital 0 in the processor to 0 volts in the circuit and the digital 1 to a set voltage, usually the voltage that the processor is using to run (commonly 5V or 3.3V).
- ▶ If you want a more nuanced more than just on/off, you need an analogue signal. For example, if you wire up a potentiometer to let you read in the position of a rotary knob, you will get a varying voltage, depending on the knob's location. Similarly, if you want to run a motor at a speed other than off or full-speed, you need to feed it with a voltage somewhere between 0V and its maximum rating.

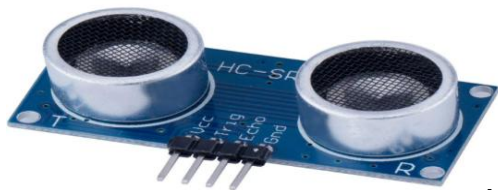
Electronics

Because computers are purely digital devices, you need a way to translate between the analogue voltages in the real world and the digital of the computer.

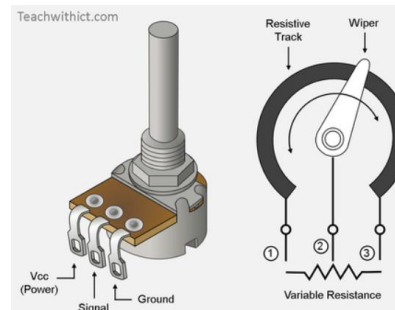
- An analogue-to-digital converter (ADC) lets you measure varying voltages.
- Microcontrollers often have a number of these converters built in.
- For more complicated sensors and modules, there are interfaces e.g. **Serial Peripheral Interface (SPI) bus and Inter-Integrated Circuit (I2C)**.
- These standardized mechanisms allow modules to communicate, so sensors or things such as Ethernet modules or SD cards can interface to the microcontroller.

Sensors

- ▶ Pushbuttons and switches, which are probably the simplest sensors, allow some user input. Potentiometers (both rotary and linear) and rotary encoders enable you to measure movement.
- ▶ Sensing the environment is another easy option. Light-dependent resistors (LDRs) allow measurement of ambient light levels, thermistors and other temperature sensors allow you to know how warm it is, and sensors to measure humidity or moisture levels are easy to build.



Ultrasonic sensor



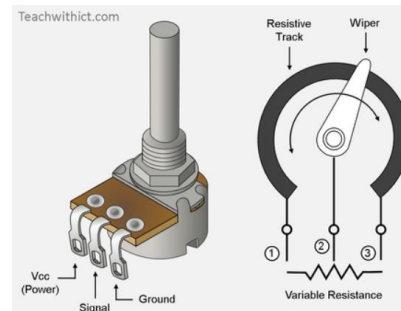
Potentiometers

Pressure sensor



Sensors

- ▶ A potentiometer is a three-terminal resistor with a sliding or rotating contact that forms an adjustable voltage divider . If only two terminals are used, one end and the wiper, it acts as a variable resistor or rheostat.
- ▶ Microphones obviously let you monitor sounds and audio, but [piezo elements](#) (used in certain types of microphones) can also be used to respond to vibration.
- ▶ Distance-sensing modules, which work by bouncing either an infrared or ultrasonic signal off objects, are readily available and as easy to interface to as a potentiometer



Potentiometers

ACTUATORS

[a device that causes a machine or other device to operate]

- ▶ One of the simplest and yet most useful actuators is light, because it is easy to create electronically and gives an obvious output.
- ▶ **Light-emitting diodes (LEDs)** typically come in red and green but also white and other colors.
- ▶ **RGB LEDs** have a more complicated setup but allow you to mix the levels of red, green, and blue to make whatever color of light you want.
- ▶ More complicated visual outputs also are available, such as LCD screens to display text or even simple graphics



ACTUATORS

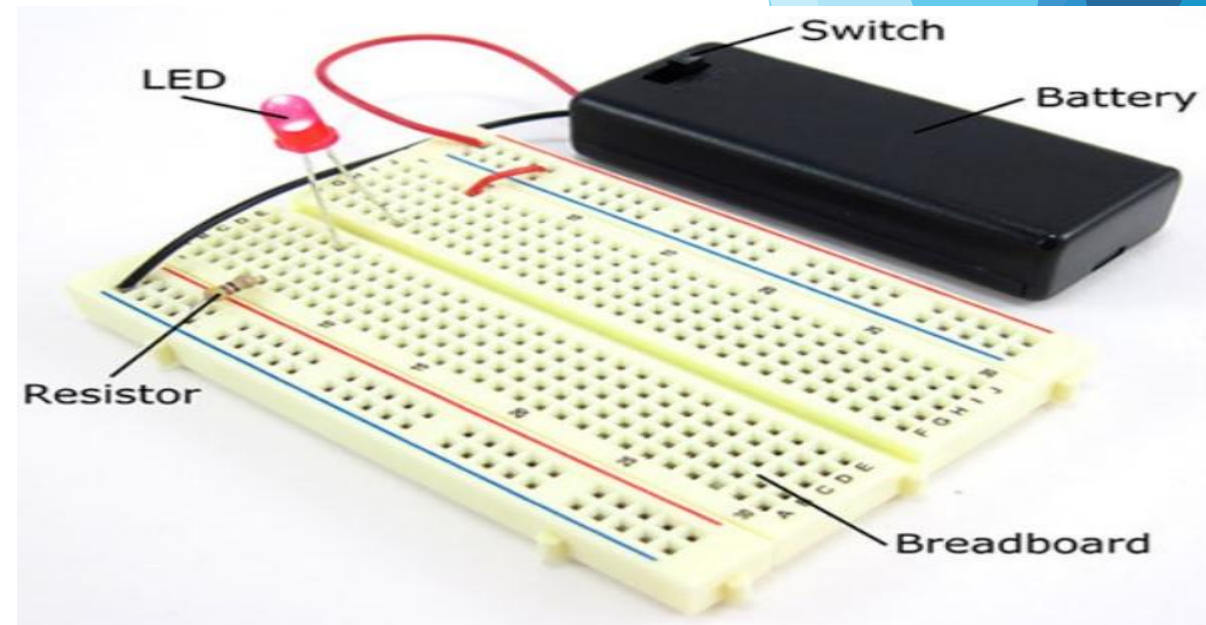
- ▶ Piezo elements, as well as responding to vibration, can be used to create it, so you can use [a piezo buzzer to create simple sounds and music](#). Alternatively, you can wire up outputs to speakers to create more complicated synthesized sounds.
- ▶ [Solenoids can be used to create a single, sharp pushing motion](#), which could be useful for pushing a ball off a ledge or tapping a surface to make a musical sound.
- ▶ More complicated again are motors. Stepper motors can be moved in steps, as the name implies. Usually, a fixed number of steps perform a full rotation.

ACTUATORS

- ▶ DC motors simply move at a given speed when told to. Both types of motor can be one-directional or move in both directions.
- ▶ Alternatively, if you want a motor that will turn to a given angle, you would need a servo. Although a servo is more controllable, it tends to have a shorter range of motion,

SCALING UP THE ELECTRONICS

- ▶ From the perspective of the electronics, the starting point for prototyping is usually a “breadboard”. This lets you push-fit components and wires to make up circuits without requiring any soldering and therefore makes experiment easy.
- ▶ When you’re happy with how things are wired up, it’s common to solder the components onto some protoboard, which may be sufficient to make the circuit more permanent and prevent wires from going astray.



SCALING UP THE ELECTRONICS

- ▶ If you need to make many copies of the circuit ,or if you want a professional finish , you can turn your circuit into PCB (printed circuit board).
- ▶ This makes it easier to build up the circuit because the position of each component will be labelled, there will be holes only where the components go, and there will be less chance of short circuits because the tracks between components will be protected by the solder resist.



The PCB.

EMBEDDED COMPUTING BASICS

MICROCONTROLLERS: Internet of Things devices take advantage of more tightly integrated and miniaturized solutions—from the most basic level of microcontrollers to more powerful system-on-chip (SoC) modules. These systems combine the processor, RAM, and storage onto a single chip, which means they are much more specialised, smaller than their PC equivalents, and also easier to build into a custom design.

- ▶ Unlike the market for desktop computer processors, which is dominated by two manufacturers (Intel and AMD), the microcontroller market consists of many manufacturers.
- ▶ A better comparison is with the automotive market. so there are lots of microcontroller **manufacturers (Atmel, Microchip, NXP, Texas Instruments, to name a few)**, each with a range of chips for different applications.

EMBEDDED COMPUTING BASICS

- ▶ The ubiquitous Arduino platform is based around Atmel's AVR ATmega family of microcontroller chips. The on-board inclusion of an assortment of **GPIO pins and ADC circuitry means that microcontrollers** are easy to wire up to all manner of sensors, lights, and motors.
- ▶ Because the devices using them are focused on performing one task, they can dispense with most of what we would term an operating system, resulting in a simpler and much slimmer code footprint than that of a SoC or PC solution.

SYSTEM-ON-CHIPS

- ▶ In between the low-end microcontroller and a full-blown PC sits the SoC (for example, the BeagleBone or the Raspberry Pi). Like the microcontroller, these SoCs combine a processor and a number of peripherals onto a single chip but usually have more capabilities.
- ▶ The processors usually range from a few hundred megahertz, nudging into the gigahertz for top-end solutions, and include RAM measured in megabytes rather than kilobytes. Storage for SoC modules tends not to be included on the chip, with SD cards being a popular solution.



BeagleBone



Raspberry Pi

CHOOSING YOUR PLATFORM

- ▶ The platform you choose depends on the particular blend of price, performance, and capabilities that suit what you're trying to achieve.
- ▶ And just because you settle on one solution, that doesn't mean somebody else wouldn't have chosen a completely different set of options to solve the same problem.
- ▶ Consider the following while choosing a platform
 - ▶ Processor speed
 - ▶ RAM
 - ▶ Networking
 - ▶ USB
 - ▶ Power consumption
 - ▶ Interfacing with sensors and other Circuit
 - ▶ Physical size and Form Factor

CHOOSING YOUR PLATFORM

Processor Speed

- ▶ The processor speed, or clock speed, of your processor tells you how fast it can process the individual instructions in the machine code for the program it's running.
- ▶ Naturally, a faster processor speed means that it can execute instructions more quickly. The clock speed is still the simplest proxy for raw computing power, but it isn't the only one. E.g. millions of instructions per second (MIPS)
- ▶ Microcontrollers tend to be clocked at speeds in the tens of MHz, whereas (System on Chips) SoCs run at hundreds of MHz or possibly low GHz.

CHOOSING YOUR PLATFORM

Processor Speed

- ▶ Example: If your project doesn't require heavy weight processing—for example, if it needs only networking and fairly basic sensing—then some sort of micro controller will be fast enough. If your device will be crunching lots of data—for example, processing video in real time—then you'll be looking at a SoC platform
- ▶ Example: Some processors may lack hardware floating point calculation. So if the code involves a lot of complicated mathematics , a by-the-numbers slower processor with the hardware floating point support could be faster than a slightly higher performance processor without it.

CHOOSING YOUR PLATFORM

RAM

- ▶ RAM provides the working memory for the system.
- ▶ If you have more RAM, you may be able to do more things or have more flexibility over your choice of coding algorithm.
- ▶ It is difficult to give exact guidelines to the amount of RAM you will need, as it will vary from project to project.
- ▶ However, microcontrollers with less than 1KB of RAM are unlikely to be of interest, and if you want to run standard encryption protocols, you will need at least 4KB, and preferably more.
- ▶ For SoC(system on chip) boards, particularly if you plan to run Linux as the operating system, we recommend at least 256MB

CHOOSING YOUR PLATFORM

Networking

- ▶ How your device connects to the rest of the world is a key consideration for Internet of Things products. Wired Ethernet is often the simplest for the user—generally plug and play—and cheapest, but it requires a physical cable.
- ▶ Wireless solutions obviously avoid that requirement but introduce a more complicated configuration.
- ▶ WiFi is the most widely deployed to provide an existing infrastructure for connections, but it can be more expensive and less optimized for power consumption than some of its competitors.

CHOOSING YOUR PLATFORM

Networking

- ▶ Other short-range wireless can offer better power-consumption profiles or costs than WiFi but usually with the trade-off of lower bandwidth. ZigBee is one such technology, aimed particularly at sensor networks and scenarios such as home automation.
- ▶ The recent Bluetooth LE protocol (also known as Bluetooth 4.0) has a very low power-consumption profile similar to ZigBee's and could see more rapid adoption due to its inclusion into standard Bluetooth chips included in phones and laptops.
- ▶ For remote or outdoor deployment, little beats simply using the mobile phone networks. For low-bandwidth, higher-latency communication, you could use something as basic as SMS; for higher data rates, you will use the same data connections, like 3G, as a smartphone.

CHOOSING YOUR PLATFORM

USB

- ▶ If your device can rely on a more powerful computer being nearby, tethering to it via USB can be an easy way to provide both **power and networking**.
- ▶ You can buy some of the microcontrollers in versions which include support for USB, so choosing one of them **reduces the need for an extra chip in your circuit**.
- ▶ Instead of the microcontroller presenting itself as a device, some can also act as the USB “host”.
- ▶ This configuration lets you connect items that would normally expect to be connected to a computer—devices such as phones, for example, using the Android ADK, additional storage capacity, or WiFi dongles.
- ▶ Devices such as WiFi dongles often depend on additional software on the host system, such as networking stacks, and so are better suited to the more computer-like option of SoC



CHOOSING YOUR PLATFORM

power consumption

- ▶ Faster processors are often more power hungry than slower ones.
- ▶ For devices which might be portable or rely on an unconventional power supply (batteries, solar power) depending on where they are installed, power consumption may be an issue.
- ▶ Even with access to mains electricity, the power consumption may be something to consider because lower consumption may be a desirable feature.

CHOOSING YOUR PLATFORM

power consumption

- ▶ However, processors may have a minimal power-consumption sleep mode.
- ▶ This mode may allow you to use a faster processor to quickly perform operations and then return to low-power sleep.
- ▶ Therefore, a more powerful processor may not be a disadvantage even in a low-power embedded device

CHOOSING YOUR PLATFORM

Interfacing with Sensors and Other Circuitry

- ▶ your device needs to interact with—either sensors to gather data about its environment; or motors, LEDs, screens, and so on, to provide output.
- ▶ You could connect to the circuitry through some sort of peripheral bus—SPI and I2C being common ones—or through ADC or DAC modules to read or write varying voltages; or through generic GPIO pins, which provide digital on/off inputs or outputs.
- ▶ Different microcontrollers or SoC (system on chip) solutions offer different mixtures of these interfaces in differing numbers.

CHOOSING YOUR PLATFORM

Physical Size and Form Factor

- ▶ With the traditional through-whole design, most commonly used for homemade circuits, the legs of the chip are usually spaced at 0.1 intervals.
- ▶ Even if your chip has relatively few connections to the surrounding circuit—16 pins is nothing for such a chip—you will end up with over 1.5" (~4cm) for the perimeter of your chip.
- ▶ You can pack the legs closer together with **surface-mount technology** because it doesn't require holes to be drilled in the board for connections.
- ▶ Combining that with the trick of hiding some of the connections on the underside of the chip means that it is possible to use the complex designs without resorting to PCBs the size of a table.

CHOOSING YOUR PLATFORM

Physical Size and Form Factor

- ▶ The limit to the size that each connection can be reduced to is then governed by the capabilities and tolerances of your manufacturing process.
- ▶ Some surface-mount designs are big enough for home-etched PCBs and can be hand-soldered.
- ▶ Others require professionally produced PCBs and accurate pick-and-place machines to locate them correctly.
- ▶ Due to these trade-offs in size versus manufacturing complexity, many chip designs are available in a number of different form factors, known as packages.
- ▶ This lets the circuit designer choose the form that best suits his particular application.

CHOOSING YOUR PLATFORM

Physical Size and Form Factor

- ▶ All three chips pictured in the following figure provide identical functionality because they are all AVR ATmega328 microcontrollers.
- ▶ The one on the left is the through-hole package, mounted here in a socket so that it can be swapped out without soldering.
- ▶ The two others are surface mount, in two different packages, showing the reduction in size but at the expense of ease of soldering.



Through-hole versus surface-mount ATmega328 chips.

Choosing Platform

ARDUINO

- ▶ These days the Arduino project covers a number of microcontroller boards, but its birth was in Ivrea in Northern Italy in 2005.
- ▶ A group from the Interaction Design Institute Ivrea (IDII) wanted a board for its design students to use to build interactive projects. An assortment of boards was around at that time, but they tended to be expensive, hard to use, or both.

DEVELOPING ON THE ARDUINO

- ▶ **Integrated Development Environment:** Most Arduino projects consist of a single file of code, so you can think of the IDE mostly as a simple file editor. The controls that you use the most are those to check the code (by compiling it) or to push code to the board.
- ▶ **Pushing Code:** Connecting to the board should be relatively straightforward via a USB cable. When your setup is correct, the process of pushing code is generally simple: first, the code is checked and compiled, with any compilation errors reported to you. If the code compiles successfully, it gets transferred to the Arduino and stored in its flash memory. At this point, the Arduino reboots and starts running the new code.
- ▶ **Operating System:** The Arduino doesn't, by default, run an OS as such, only the bootloader, which simplifies the code-pushing process described previously. It is, however, possible to upload an OS to the Arduino, usually a lightweight real-time operating system (RTOS) such as FreeRTOS/DuinOS. The main advantage of one of these operating systems is their built-in support for multitasking.



An Arduino Ethernet board, plugged in, wired up to a circuit and ready for use.

DEVELOPING ON THE ARDUINO

- ▶ **Language:** The language usually used for Arduino is a slightly modified dialect of C++ derived from the Wiring platform. It includes some libraries used to read and write data from the I/O pins provided on the Arduino and to do some basic handling for “interrupts”. **The code needs to provide only two routines:**
- ▶ **Setup():** This routine is run once when the board first boots. You could use it to set the modes of I/O pins to input or output or to prepare a data structure which will be used throughout the program.
- ▶ **Loop():** This routine is run repeatedly in a tight loop while the Arduino is switched on. Typically, you might check some input, do some calculation on it, and perhaps do some output in response.

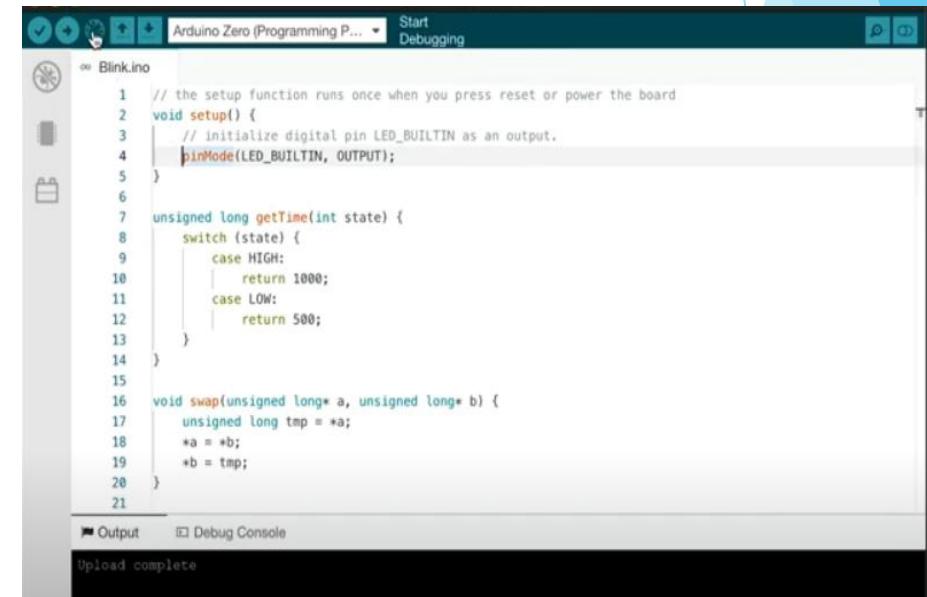
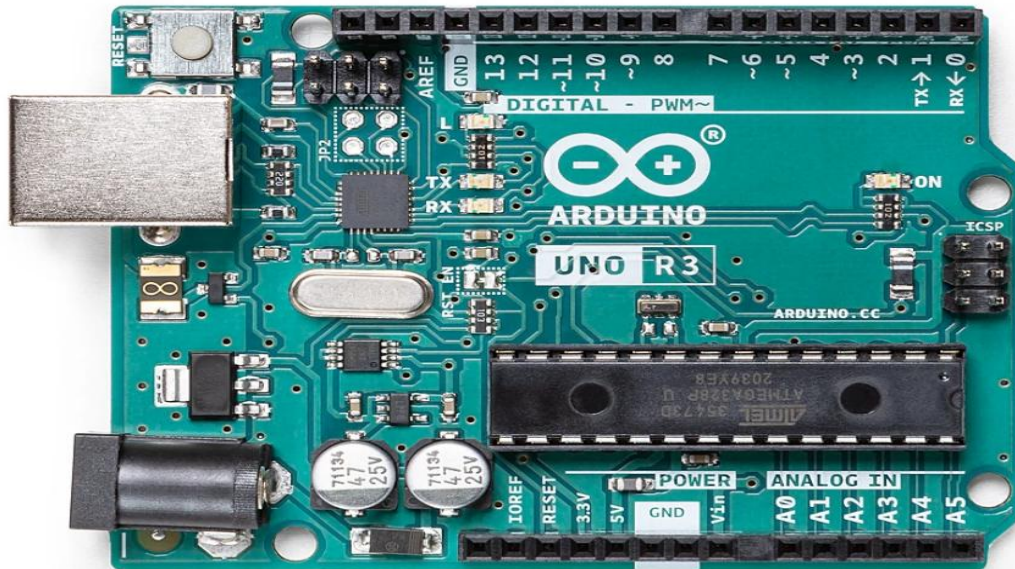
DEVELOPING ON THE ARDUINO

Blinking a single LED

```
// Pin 13 has an LED connected on most Arduino boards. // give it a name: int led = 13;
// the setup routine runs once when you press reset:
void setup() {
  // initialize the digital pin as an output.
  pinMode(led, OUTPUT);
}
// the loop routine runs over and over again forever:
void loop() {
  digitalWrite(led, HIGH); // turn the LED on
  delay(1000); // wait for a second
  digitalWrite(led, LOW); // turn the LED off
  delay(1000); // wait for a second }
```

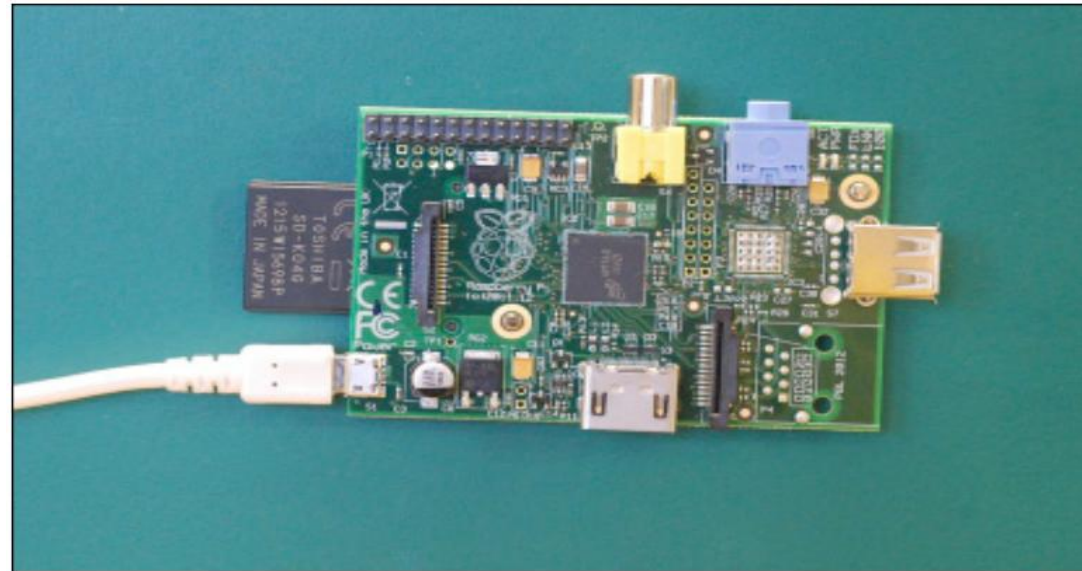

DEVELOPING ON THE ARDUINO

- **Debugging:** Because C++ is a compiled language, a fair number of errors, such as bad syntax or failure to declare variables, are caught at compilation time. Because this happens on your computer, you have ample opportunity to get detailed and possibly helpful information from the compiler about what the problem is.



RASPBERRY PI

- ▶ The Raspberry Pi is effectively a computer that can run a real, modern operating system, communicate with a keyboard and mouse, talk to the Internet, and drive a TV/monitor with high-resolution graphics.



A Raspberry Pi Model B board. The micro USB connector only provides power to the board; the USB connectivity is provided by the USB host connectors (centre-bottom and centre-right).

DEVELOPING ON THE RASPBERRY PI

- ▶ **Operating System** Although many operating systems can run on the Pi, we recommend using a popular Linux distribution, such as
- ▶ Raspbian: Released by the Raspbian Pi Foundation, **Raspbian** is a distro **based on Debian**. This is the default “official” distribution and is certainly a good choice for general work with a Pi.
- ▶ Occidentalis: This is Adafruit’s customised Raspbian. Unlike Raspbian, the distribution assumes that you will use it “headless”—not connected to keyboard and monitor—so you can connect to it remotely by default.

DEVELOPING ON THE RASPBERRY PI

Programming Language

- ▶ There is some guidance from the Foundation, which suggests Python as a good language for educational programming (and indeed the name “Pi” comes initially from Python)

Python will almost certainly make the following tasks easier:

- ▶ Handling strings of character data
- ▶ Completely avoiding having to handle memory management (and bugs related to it)
- ▶ Making calls to Internet services and parsing the data received
- ▶ Connecting to databases and more complex processing
- ▶ Abstracting common patterns or complex behaviors

Contrast Python with C++

Python, as with most high-level languages, compiles to relatively large (in terms of memory usage) and slow code, compared to C++.

- ▶ The former is unlikely to be an issue; the Pi has more than enough memory. The speed of execution may or may not be a problem
- ▶ Python is likely to be “fast enough” for most tasks, and certainly for anything that involves talking to the Internet, the time taken to communicate over the network is the major slowdown.
- ▶ However, if the electronics of the sensors and actuators you are working with require split-second timing, Python might be too slow.
- ▶ This is by no means certain; if Bubblino starts blowing bubbles a millisecond later, or the DoorBot unlocks the office a millisecond after you scan your RFID card to authenticate, this delay may be acceptable and not even noticeable.

Contrast Python with C++

Python handles memory management automatically.

Because handling the precise details of memory allocation is notoriously fiddly, automatic memory management generally results in fewer bugs and performs adequately.

However, this automatic work has to be scheduled in and takes some time to complete.

Depending on the strategy for garbage collection, this may result in pauses in operation which might affect timing of subsequent events.

Contrast Python with C++

Linux itself arguably has some issues for “real-time” use.

Due to its being a relatively large operating system, with many processes that may run simultaneously, precise timings may vary due to how much CPU priority is given to the Python runtime at any given moment.

This hasn't stopped many embedded programmers from moving to Linux, but it may be a consideration for your case.

Contrast Python with C++

An Arduino runs only the one set of instructions, in a tight loop, until it is turned off or crashes.

The Pi constantly runs a number of processes.

If one of these processes misbehaves, or two of them clash over resources may cause problems that are entirely unrelated to your code.

This is unlikely but may result in occasional, possibly intermittent, issues which are hard to identify and debug.

SOME NOTES ON THE HARDWARE

- ▶ The Raspberry Pi has **8 GPIO pins**
- ▶ Unlike those in the Arduino, the pins in the Raspberry Pi **aren't individually labelled.**, you can connect individual pins using a female jumper lead onto a breadboard.
- ▶ The block of pins provides both **5V and 3.3V outputs.**
- ▶ However, the GPIO pins themselves are **only 3.3V tolerant.** The Pi doesn't have any over-voltage protection, so you are at **risk of breaking the board if you supply a 5V input.**
- ▶ Use an external breakout board that has this kind of protection.
- ▶ Note: **that the Raspberry Pi doesn't have any analogue inputs (ADC), which means that options to connect it to electronic sensors are limited.**
- ▶ To get readings from light-sensitive photocells, temperature sensors, potentiometers, and so on, you **need to connect it to an external ADC via the SPI bus.**

Comparison and recommendation

	Arduino	Gadgeteer	Raspberry Pi
Model	R3	FEZ Spider	Model B
Price	\$30	\$120	\$35
Processor	ATMega 328	ARM7	ARM11
Clock Speed	16MHz	72MHz (168MHz for FEZ Cerberus)	700MHz
RAM	2KB	16MB	256MB
Flash	32KB	4.5MB	(SD Card)
Min Power	42mA (0.3W)	160mA	700mA (3.5W)
Dev IDE	Arduino Tool	Visual Studio	IDE, Scratch, Squeak/Linux
	Low-level C++	Managed code Concise code Real-time debugging Excellent modular concept, very good for code and hardware re-use.	being programmed in many different languages
	run one program at a time	run one program at a time	capable of running multiple programs at the same time
			Ethernet, 2USB HDMI, Composite

Recommendation

- ▶ For applications minimizing size -> [Arduino](#)
 - There are very small Arduino embedded systems for making very tiny little gadget.
- ▶ For battery powered applications -> [Arduino](#)
 - Uses the least power of the bunch.
 - Work with a wide range of input voltages (so support a variety of different types of batteries).
- ▶ For applications that interface to external sensors -> [Arduino](#), [Gadgeteer](#)
 - Arduino: lots of external (cheap) sensors
 - Gadgeteer: many slots available for connecting multiple sensors easily.
- ▶ For applications that connect to the internet -> [Gadgeteer](#), [Raspberry Pi](#)
 - Gadgeteer: Full TCP/IP Stack with SSL, HTTP, TCP, UDP, DHCP
 - Raspberry Pi: the Linux OS has many components built-in that provide rather advanced networking capabilities.
- ▶ For applications that use a graphical user interface -> [Raspberry Pi](#)
 - It has an HDMI output.
 - A fully functional computer with graphical user interface.

References

- ▶ 1. Book - Designing Internet of Things (chapter 4 and chapter 5)
By Adrian Ecewen & Hakim

Question



Thank you

